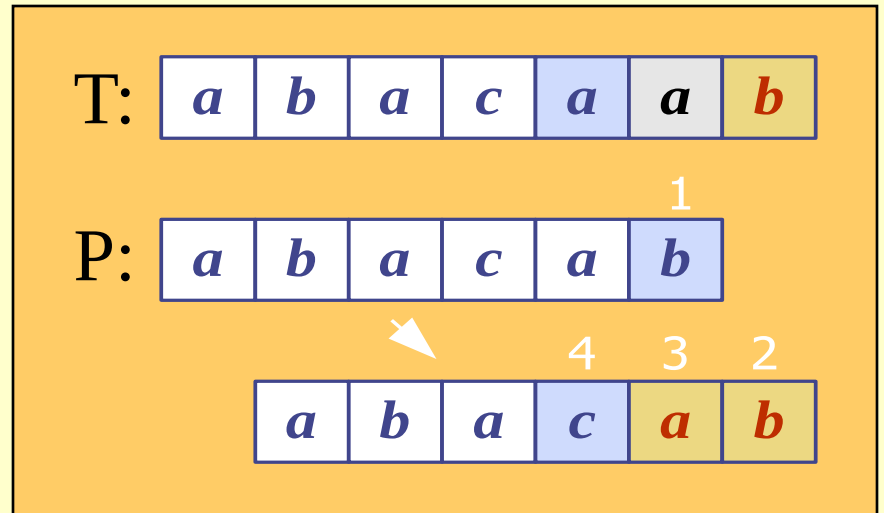


§ 4.3 Pattern Matching (模式匹配)



Overview

1. What is Pattern Matching?
2. The Brute Force Algorithm 直接算法
3. The Knuth-Morris-Pratt Algorithm KMP 算法
- *4. The Boyer-Moore Algorithm BM 算法
5. More Information

1. What is Pattern Matching?

什么是模式匹配

- 定义:
 - 给定一个文本串T和一个模式串P，找到模式在文本中的位置
 - T: “the rain in spain stays mainly on the plain”
 - P: “n st”
- 应用:
 - 文字编辑器, Web搜索 (e.g. Google), 图像分析

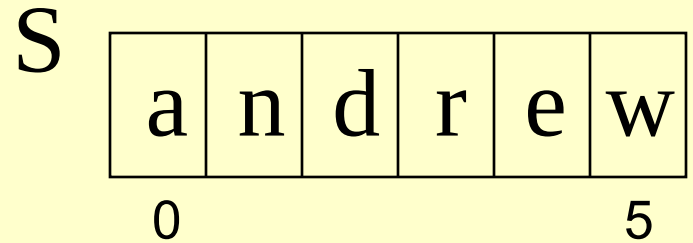
字符串相关概念

- Assume S is a string of size m .

假定串 S 的长度为 m

- 子串 A *substring* $S[i .. j]$ of S is the string fragment between indexes i and j .
- 前缀 A *prefix* of S is a substring $S[0 .. i]$
- 后缀 A *suffix* of S is a substring $S[i .. m-1]$
 - i is any index between 0 and $m-1$

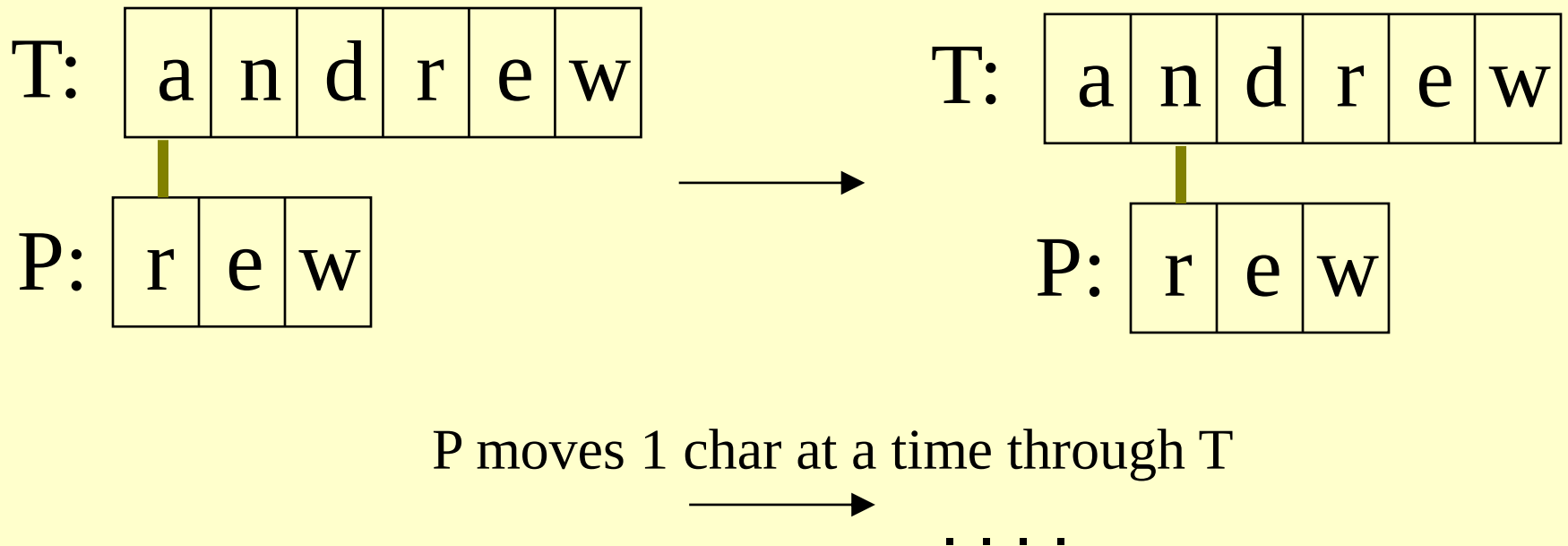
Examples



- 子串 $S[1..3] == \text{"ndr"}$
- S所有可能前缀:
 - "andrew", "andre", "andr", "and", "an", "a"
- S所有可能后缀:
 - "andrew", "ndrew", "drew", "rew", "ew", "w"

2. The Brute Force Algorithm 直接算法

- 检查每一个位置



直接算法

Algorithm *BruteForceMatch*(T, P)

Input: text T of size n and pattern P of size m

Output: starting index of a substring of T equal to P or -1 if no such substring exists

1. **for** $i \leftarrow 0$ **to** $n - m$
2. //test shift i of the pattern
3. $j \leftarrow 0$
4. **while** $j < m \wedge T[i + j] = P[j]$
5. $j \leftarrow j + 1$
6. **if** $j = m$
7. **return** i //match at i
8. **else**
9. **break while loop** //mismatch
10. **return** -1 //no match anywhere

Return模式位置或-1

Analysis分析

- 直接算法最坏情况代价为 $O(mn)$
- 大多数普通文本 $O(m+n)$, 很快了.

- 当不同字符数多时，字母表大， 直接算法较快
 - e.g. A..Z, a..z, 1..9, etc.
- 字母表小时， 算法较慢
 - e.g. 0, 1 (as in binary files, image files, etc.)

Example of a worst case:

- T: "aaaaaaaaaaaaaaaaaaaaaaaaaaaaah"
- P: "aaah"

Example of a more average case:

- T: "a string searching example is standard"
- P: "store"

Experiment 4-2(a) 模式匹配的直接算法

问题:

1. 从文件读取文本 *str* 和模式 *pattern* 数据;
2. 返回匹配串的第一个位置 ;
3. 用 `strlen( )` 获得长度:

lenStr = strlen(str) .

Experiment 4-2(b) 模式匹配的直接算法

问题:

1. 从文件读取文本 *str* 和模式 *pattern* 数据;
2. 返回 *str* 中的 **所有** 匹配位置 ;
3. Use a loop to call function of finding substring :
while ((lenStr - *nextLocation)>=lenPattern) { }

3. KMP Algorithm

- Knuth-Morris-Pratt (KMP) 从左向右移动匹配模式；
- 基于部分匹配表(Partial Match Table)的数组，智能移动模式。

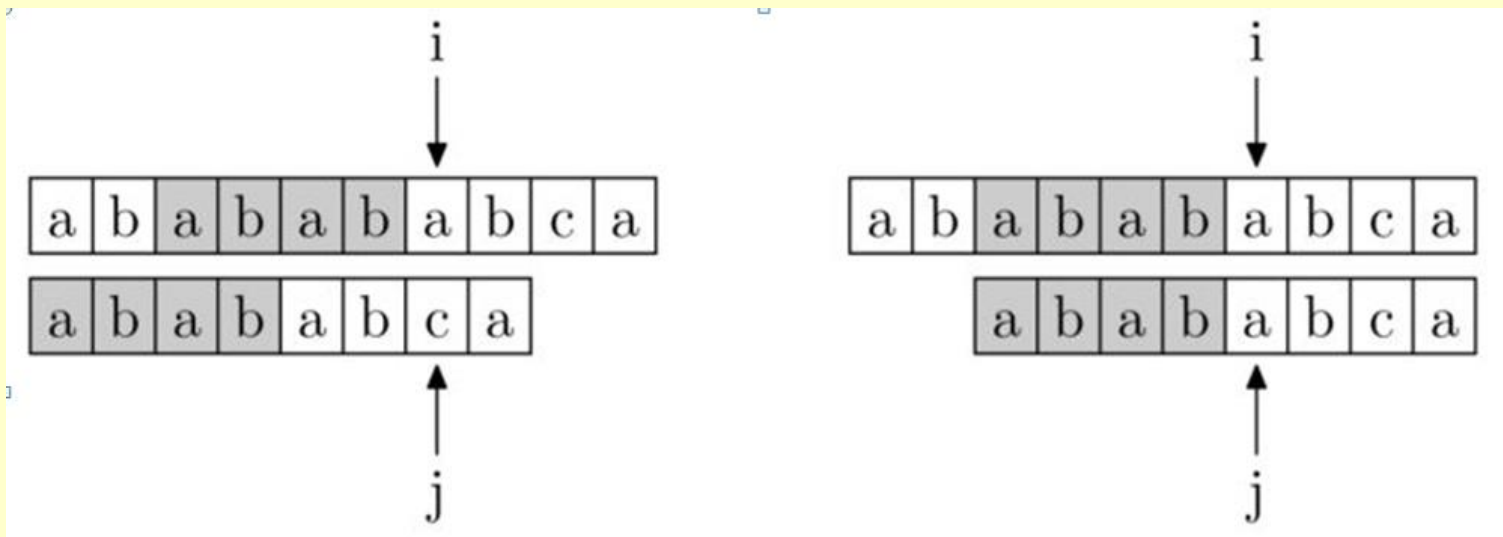
PMT: 字符串的前缀集合与后缀集合的交集中最长元素的**长度**。

- “ababa”，前缀集合为{“a”, “ab”, “aba”, “abab”}, 后缀集合为{“baba”, “aba”, “ba”, “a”}, 两个集合的交集为{“a”, “aba”}, 最长的元素为“aba”，长度为3。(不包括字符串本身)
- 当前**部分串**中相同前缀与后缀**最大长度**
- 字符串 “abababca”的PMT

char:	a	b	a	b	a	b	c	a
index:	0	1	2	3	4	5	6	7
value:	0	0	1	2	3	4	0	1

匹配

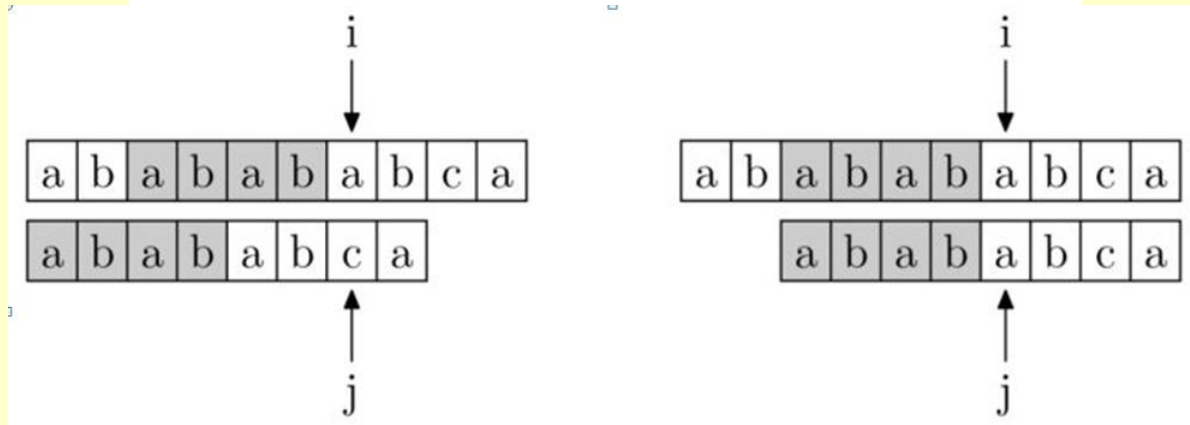
- 主字符串“ababababca”，模式字符串“abababca”。
- 如果模式串在 j 处、主字符串在 i 位字符不匹配，则主字符串从 $i-j$ 到 i 与模式字符串的 0 到 j 是完全相同的。因此，字符串中 i 指针之前的 $\text{PMT}[j-1]$ 位就一定与模式字符串的第 0 位至第 $\text{PMT}[j-1]$ 位是相同的。 $(\text{PMT}[j-1]$ 定义)
- 对于“ababab”，其前缀集合与后缀集合的交集的最长元素为“abab”，长度为4。所以主字符串中 i 指针之前的 4 位一定与模式字符串的第 0 位至第 4 位是相同的，即长度为 4 的后缀与前缀相同。可以将这些字符段的比较省略掉，保持 i 指针不动，然后将 j 指针指向模式字符串的 $\text{PMT}[j-1]$ 位。



- 如果是在 j 位失配，那么影响 j 指针回溯的位置其实是第 $j-1$ 位的 PMT 值；
- 将PMT数组向后偏移一位，得到next数组。
- next数组的含义：在子串的第 j 个字符与主串发生失配时，则跳到子串的 $next[j]$ 位置重新与主串当前位置进行比较。
- 第0位的值，将其设成为-1。

char:	a	b	a	b	a	b	c	a
index:	0	1	2	3	4	5	6	7
pmt:	0	0	1	2	3	4	0	1
next:	-1	0	0	1	2	3	4	0

https://blog.csdn.net/x_1998



移动位数为
 $j - next[j]$
 $= 6 - 4$

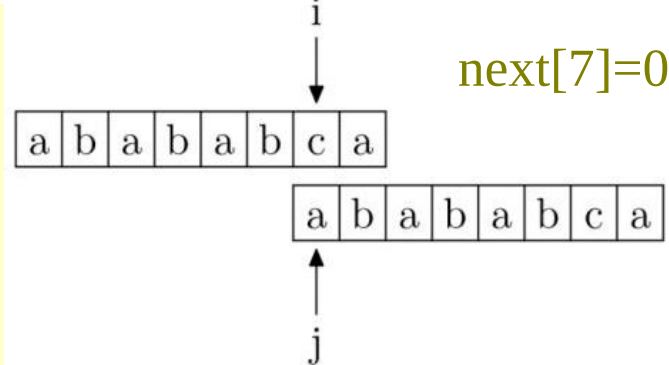
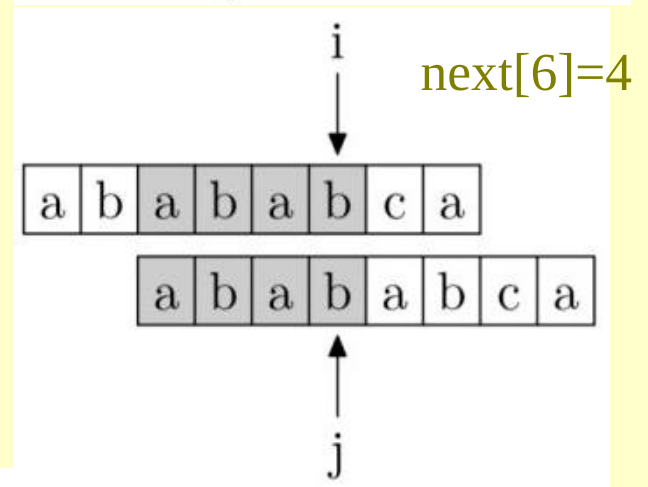
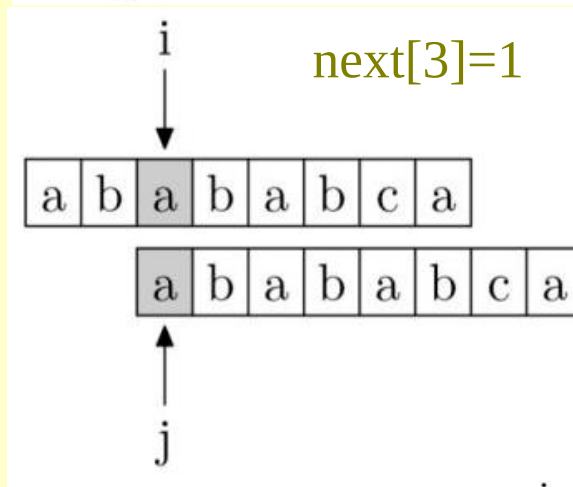
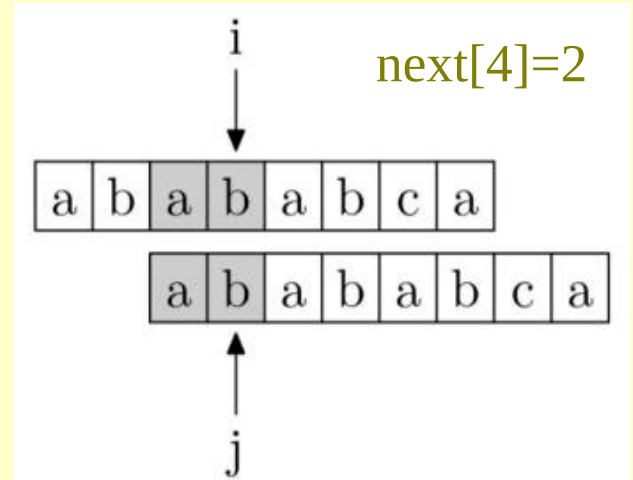
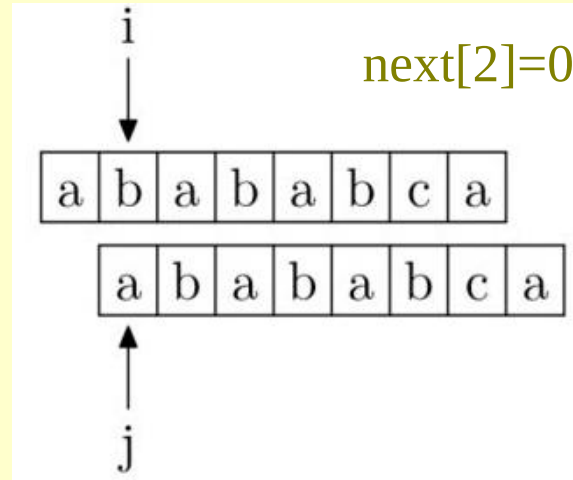
```
int KMP(char * t, char * p)
{  int i = 0, j = 0;
    while (i < strlen(t) && j < strlen(p)) {
        if (j == -1 || t[i] == p[j]) {
            i++;
            j++;
        }
        else  j = next[j]; //失配时j的新值, 移动位数为旧j-next[j]
    }
    if (j == strlen(p))  return i - j;
    else  return -1;
}
```


next[0]=-1 next[1]=0 (不包括字符串本身)

求解next数组

- 看成字符串匹配的过程，即以模式字符串为主字符串，以模式字符串的前缀为目标字符串，一旦字符串匹配成功，那么当前的next值就是匹配成功的字符串的长度
- 从模式字符串的第一位(不包括第0位)开始对自身进行匹配运算。在任一位置，能匹配的最长长度就是当前位置的next值。

i最大值为7



next[5]=?

```
void getNext(char * p, int * next)
{
    int i = 0, j = -1;
    next[0] = -1;
    while (i < strlen(p)){
        if (j == -1 || p[i] == p[j]){
            ++i;
            ++j;
            next[i] = j; //连续匹配上的个数
        }
        else j = next[j]; //失配时j的新值
    }
}
```

Example

T:

<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>c</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>b</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------

P:

1	2	3	4	5	6
<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>

j 0...5

7					
<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>

8	9	10	11	12	
<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>

13					
<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>

14	15	16	17	18	19
<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>

<i>index</i>	1	2	3	4	5
<i>next</i>	0	0	1	0	1

练习：当P为以下字符串时，计算二个数组

pmt[]

next[]

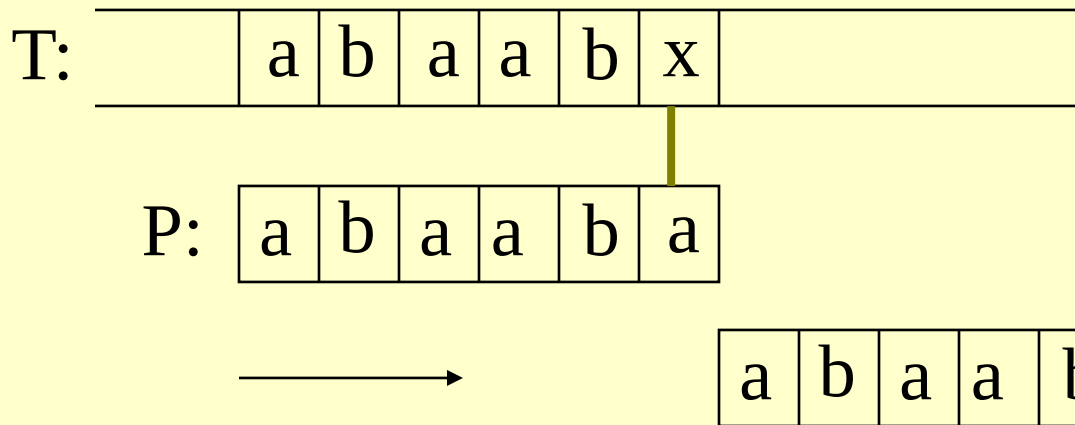
(1) a b a b

(2) a b a a b a

(3) a b a c a b

KMP优缺点

- KMP 最优时间: $O(m+n)$
- 算法不需向后退, 适合外存文件;
- 不适合于字母表太大情况;
- 基本算法不考虑失配情况。



Basic KMP
does **not** do this.

➤ KMP算法在最坏情况下均具有线性的查找时间; 还有一些算法如BM算法、Sunday算法。

Experiment 4-3 KMP Algorithm for Pattern Matching

Problems:

1. 返回 **text** 中 **pattern** 出现的所有位置;
2. 文件中读取, 测试不同的数据与模式;
3. 比较直接算法、KMP 算法匹配代价, 用比较次数来计算。

力扣 28. 实现 strStr()

力扣 459. 重复的子字符串

1、已知串S='ababaaababaa'，其next数组为（ ）。

A. -1,0,1,2,3,4,5,6,7,8,8,8 B. -1,0,1,0,1,0,0,0,0,1,0,1

C. -1,0,0,1,2,3,1,1,2,3,4,5 D. -1,0,1,2,-1,0,1,2,1,1,2,3

2、【2015统考真题】已知串S='abaabaabacacaababbcc'，模式串t为'abaabc'。采用KMP算法进行匹配，第一次出现失配 ($s[i] \neq s[j]$) 时， $i=j=5$ ，则下次开始匹配时，i和j的值分别是（ ）。

A. $i=1, j=0$ B. $i=5, j=0$ C. $i=5, j=2$ D. $i=6, j=2$

3、【2019统考真题】设主串T='abaabaabcabaabc'，模式串S为'abaabc'。采用KMP算法进行匹配，到匹配成功为止，在匹配过程中进行的单个字符间的比较次数是（ ）。

A. 9 B. 10 C. 12 D. 15

4、已知串S='aaab'，其next数组值为（ ）。

A. -1,0,1,2 B. -1,0,0,1 C. -1,1,2,0 D. 0,1,0,0

5、已知串S='ababaaababaa'，其next数组值为（ ）。

A. -1,0,1,2,3,4,5,6,7,8,8 B. -1,0,1,0,1,0,0,0,0,1,0,1

C. -1,0,0,1,2,3,1,1,2,3,4,5 D. -1,0,1,2,-1,0,1,2,1,1,2,3,4

注意：在实际KMP算法中，为了使公式更简洁计算简单，如串的位序从1开始，则next数组可以整体加1；如果串的位序从0开始，则不需要整体加1。